

Conception automatique d'arches maçonnées autoportantes basées sur des classes d'équivalence

THÉO CANTALOUBE, Étudiant à l'ENSIMAG, Deuxième année, France

MÉLINA SKOURAS, Encadrante, Équipe ANIMA, Chercheuse à l'INRIA, France

Ce document résume les travaux menés par Théo Cantaloube dans le cadre du cours d'IRL (Introduction à la Recherche en Laboratoire) sur les arches maçonnées autoportantes basées sur des classes d'équivalence.

ACM Reference Format:

Théo Cantaloube and Mélina Skouras. 2026. Conception automatique d'arches maçonnées autoportantes basées sur des classes d'équivalence. 1, 1 (April 2026), 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>



Fig. 1. Les exemples de structures de coque en maçonnerie sont nombreux dans l'architecture. (Images : (en haut à gauche) Free-form masonry scale models, ETH Zurich, <https://block.arch.ethz.ch/brg/research/free-form-masonry-shells>, (en bas à gauche) Experimental Construction of a Free-Form Shell Structure in Masonry, David Wendland, <https://journals.sagepub.com/doi/abs/10.1260/026635109788251412?journalCode=spsa>, (à droite) Masonry construction, Michael Cockram, https://continuingeducation.bnppmedia.com/article_print.php?C=1647&L=146)

1 INTRODUCTION

Les structures de coque en maçonnerie ont été construites depuis des siècles dans le monde entier en forme d'arches, de dômes et de voûte en raison de leurs avantages tels que leur grande portée, leur légèreté et leur résistance élevée [Adriaenssens et al. 2014 [13]] (voir quelques exemples sur la Figure 1). Les formes des coques en

maçonnerie sont généralement décrites par des surfaces courbes en 3D, car l'épaisseur de ces structures est nettement inférieure à leur largeur et à leur longueur. D'un point de vue géométrique, une coque en maçonnerie est un carrelage géométrique d'une surface 3D avec un certain nombre d'éléments de coque qui sont en contact les uns avec les autres **sans chevauchement ni espace**. Pour carreliser une surface 3D de forme libre, il est probable que chaque élément de coque ait une forme unique par rapport aux autres, ce qui nécessite une fabrication sur mesure de chaque élément. Ceci représente un sur-coût de production non négligeable.

1.1 État de l'art

Rationalisation des surfaces architecturales. Avant de commencer à effectuer des opérations sur nos maillages, il est important de disposer de coques bien choisies. Typiquement, de nombreux travaux ont été menés sur la modélisation des surfaces avec des classes d'équivalence discrètes. Étant donné une topologie d'entrée fixe (les faces de la coques ne représentent que des triangles [Huard et al. 2015 [8]; Singh et Schaefer 2010 [14]] ou que des quadrilatères [Fu et al. 2010 [6]]), il s'agit là de regrouper les polygones en classes d'équivalences discrètes et d'optimiser les positions des sommets du maillage.

L'approche qui nous intéresse vraiment et sur laquelle se fondera une grande partie de ce projet a été abordée par Chen et al. [2] dans **Masonry Shell Structures with Discrete Equivalence Classes**. Leur modélisation du problème implique également regroupement et optimisation des polygones au sein d'une surface libre afin de minimiser le nombre d'éléments différents mais considère en plus de cela les angles dièdres entre les polygones, dans le but de faciliter le regroupement des éléments de coque dans une étape ultérieure.

Ceci dit, ces classes d'équivalence s'accompagnent de défauts d'assemblage physique. Effectivement, trois types d'erreurs peuvent compliquer la juxtaposition (Figure 2) : les erreurs dites "de contact" (les faces de deux éléments adjacents n'ont pas la même orientation), les erreurs dites "de gap" (les faces de deux éléments adjacents ne se touchent pas) et les erreurs dites de "overlap" (les faces de deux éléments adjacents se recouvrent). L'algorithme de Chen et al. établit une mesure sur ces erreurs et la minimise le plus possible, sans pouvoir atteindre le zéro absolu. Cela s'en ressent dans leurs vidéos de présentation, l'assembleur devant "forcer" sur les éléments imprimés par imprimante 3D pour qu'ils rentrent dans le moule comme sur la Figure 3).

Autoportance des structures de coque. Même si la construction des maillages est un domaine de recherche très actif, elle doit se compléter de considérations structurelles pour être utilisées dans un cadre architectural. Tandis que certains chercheurs n'envisagent la construction de coques qu'avec l'aide des formes autoportantes [de Goes et al. 2013 [5]; Liu et al. 2013 [10]; Miki et al. 2015 [11];

Authors' addresses: Théo Cantaloube, Étudiant à l'ENSIMAG, Deuxième année, Grenoble, France; Mélina Skouras, Encadrante, Équipe ANIMA, Chercheuse à l'INRIA, Meylan, France.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Association for Computing Machinery.

XXXX-XXXX/2026/4-ART \$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

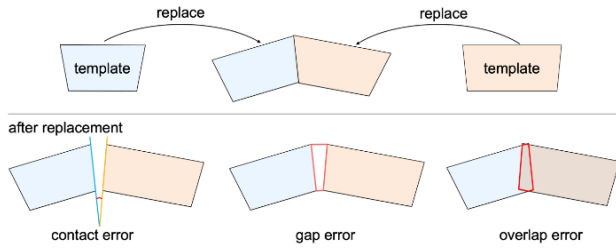


Fig. 2. Les trois types d'erreur de l'algorithme de Chen et al. (Image : Masonry Shell Structures with Discrete Equivalence Classes [Chen et al. 2023] (Fig. 7), <https://sutd-cgl.github.io/supp/Publication/projects/2023-SIGGRAPH-TileableShell/download/2023-SIGGRAPH-TileableShell.pdf>)

Panozzo et al. 2013 [12]; Tang et al. 2014 [15]; Vouga et al. 2012], d'autres préfèrent agir directement sur les structures en maçonnerie construites pour optimiser leur stabilité structurelle [Whiting et al. 2009 [18], 2012 [17]]. Les travaux établis cherchent le moins possible à s'appuyer sur les propriétés physiques des matériaux (c'est-à-dire une soutenance induite par la friction des pièces adjacentes) ou d'un liant entre les éléments, ces propriétés possédant le fâcheux désavantage de ne pas être durable dans le temps et de fluctuer au gré des intempéries, de l'érosion et de l'âge. Nous baserons une grande partie de notre algorithme d'auto-soutenance sur l'article de recherche **Design and Structural Optimization of Topological Interlocking Assembling** [Wang et al. 2019 [16]] qui construit une méthode pour quantifier le "degré d'auto-portance" d'une coque en maçonnerie. Il est à noter que cette quantification ne mesure pas seulement l'aspect statique des coques mais également leur résistance aux rotations et le poids qu'on peut leur imposer.

1.2 Objectif

Les travaux de recherche abordés dans ce papier cherchent donc à quantifier à quel point l'algorithme de Chen et al. affecte l'auto-soutenance des structures en coque en maçonnerie.



Fig. 3. L'assemblage manuel des éléments des structures en coque maçonnée peut se révéler compliqué. (Image : Masonry Shell Structures with Discrete Equivalence Classes (SIGGRAPH 2023), CGL SUTD, <https://www.youtube.com/watch?v=OJvWdn6Fcio&t=270s>)

2 VUE D'ENSEMBLE

La procédure se divise donc en trois étapes :

- **Éloigner** tous les éléments les uns des autres suivant leur convexité afin de retrouver une situation physiquement viable (i.e. sans erreur d'overlap).

- **Rapprocher** les éléments les uns des autres pour simuler un assemblage manuel de la coque en maçonnerie et respecter les contacts.

- **Vérifier** si la structure obtenue est autoportante.

Il est à noter que l'étudiante à l'Université Grenoble-Alpes Kar-lotta Kilias a déjà travaillé sur ce projet en se restreignant à des coupes en 2D des coques en maçonnerie 3D. Son algorithme est fonctionnel dans sa totalité et l'exécution est cohérente. Dans ce cours d'IRL, l'objectif est d'élargir l'algorithme à une vision 3D.

Dans ces lignes, sera désigné par *maillage* (réseau) l'ensemble de la coque en maçonnerie (mathématiquement, un maillage est représenté par (V,F) avec V l'ensemble des sommets et F l'ensemble des faces) et par *pavé* un seul des éléments qui constituent le maillage.

La manipulation des maillage 3D diffère énormément de la 2D. Une structure très utilisée dans la modélisation graphique est le maillage HalfEdge. Les maillage HalfEdge ont été conçu de manière à pouvoir visiter très facilement les faces opposées, les faces voisines, les angles contigus... Chaque arête est séparée en deux demi-arêtes, les fameuses HalfEdge, depuis lesquels on peut accéder à tous les éléments (sommets, face, HalfEdge opposé, Edge...) à proximité. Sur la Figure 4, on peut voir tous les pointeurs dont disposent les HalfEdge. Cette structure très pratique devient indispensable dans le traitement des coques en maçonnerie 3D.

L'entière du projet a été réalisé en langage C++, en se servant de 4 bibliothèques différentes :

- **Eigen** [7] : *Eigen is a C++ template library for linear algebra: matrices, vectors, numerical solvers, and related algorithms* (Eigen, TuxFamily). Elle sera utilisée pour la manipulation des matrices des sommets et des arêtes des maillages mais aussi pour la résolution des systèmes d'équations d'auto-soutenance.

- **Libigl** [9] : *libigl is a simple C++ geometry processing library* (Libigl, Github). Elle sera utilisée pour la visualisation des maillages et traitera les interactions de l'utilisateur avec les coques maçonnées (mouse ou keyboard callback).

- **OpenMesh** [1] : *The OpenMesh mesh library is a powerful tool for handling polygonal meshes* (OpenMesh, Rwth Aachen University). Elle sera utilisée pour son implémentation des maillages HalfEdge.

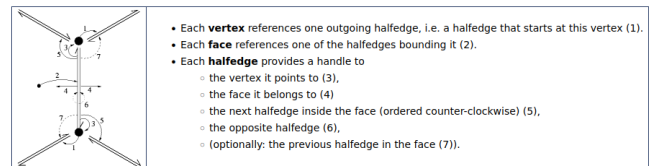


Fig. 4. Tous les liens entre les faces des maillages HalfEdge. (Image : The Halfedge Data Structure, Project OpenMesh, https://www.graphics.rwth-aachen.de/media/openmesh_static/Documentations/OpenMesh-6.2-Documentation/a00016.html)

- **TileableShell** [3] : *This repo is an implementation of [Masonry Shell Structures with Discrete Equivalence Classes]* (TileableShell, Siggraph 2023). C'est tout simplement l'algorithme de Chen et al. qui construit à partir d'une surface 3D un maillage régulier correspondant puis un maillage réduit au moyen des classes d'équivalence.

- **Bullet Physics** [4] : *Bullet Physics allows real-time collision detection and multi-physics simulation for VR, games, visual effects, robotics, machine learning* (Bullet Physics, PyBullet). Ce simulateur physique d'interaction entre objets géométriques sera utilisé uniquement pour tester les collision entre pavés.



Fig. 5. Toutes les bibliothèques utilisées (Images : (Bullet Physics) [https://fr.wikipedia.org/wiki/Bullet_\(moteur_physique\)](https://fr.wikipedia.org/wiki/Bullet_(moteur_physique)), (OpenMesh) <https://www.graphics.rwth-aachen.de/software/openmesh/>, (Eigen) https://eigen.tuxfamily.org/index.php?title=Main_Page, (Libigl) <https://libigl.github.io/tutorial/>)

3 SIMULATION

3.1 Éclatement des pavés

Premièrement, nous voulons revenir à une situation physiquement viable, c'est-à-dire une disposition où il n'y a pas de paire de pavés qui se recouvrent mutuellement (overlap error). L'idée est de les écarter d'abord en fonction de leur convexité puis de les écarter tous en fonction d'une origine fixée.

Éclatement convexe. Il est assez naturel de considérer un assemblage des pavés en fonction de leur convexité. Imaginez une coque en maçonnerie en forme de dôme à laquelle il manquerait le dernier pavé, celui à son sommet. Il serait plus facile d'essayer de l'insérer par le dessus le dôme et non par le dessous. Nous adoptons donc la convention suivante : les pavés doivent être éclatés selon leur normale dans un sens correspondant à leur convexité (convexe = déplacé vers le haut, concave = déplacé vers le bas, indéterminé = immobile). Mais pour cela, il faut savoir la calculer.

Déterminer la convexité d'une face s'effectue en deux temps. Il faut d'abord déterminer la convexité de chacun des HalfEdge en calculant l'angle entre la normale de chaque face et le vecteur entre les barycentres de la face et de la face voisine (celle de l'autre côté du HalfEdge) comme expliqué dans la Figure 6 (à noter que le cas *indéterminé* n'arrive quasiment jamais). Cela fait, on peut déterminer la convexité de chacune des faces en choisissant la convexité majoritaire dans l'ensemble de ses HalfEdges comme expliqué dans la Figure 7 (à noter qu'initialement, le postulat était qu'une face n'avait de convexité que si l'entière de ses HalfEdge avait cette même convexité. Cependant, trop de faces avaient une convexité

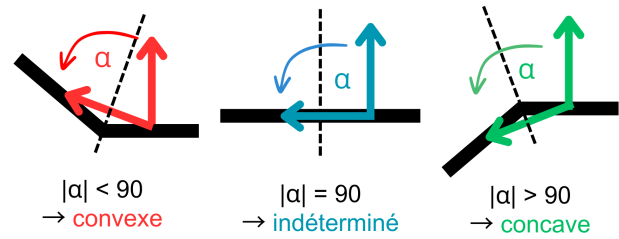


Fig. 6. Détermination de la convexité des HalfEdge.

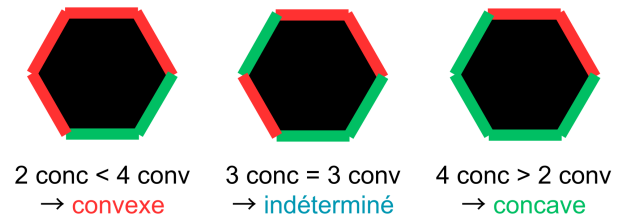


Fig. 7. Détermination de la convexité des Faces. Les contours rouges représentent des HalfEdges convexes et les contours verts représentent des HalfEdges concaves.

indéterminée donc il a fallu élargir la notion de convexité en prenant la majorité).

Éclatement à l'origine. Cependant, ce premier éclatement n'est pas complètement satisfaisant. En effet, si deux faces indéterminées sont adjacentes (ce qui est loin d'être rare), elles ne seront pas déplacées selon leur normale et peuvent toujours présenter une situation physiquement impossible à cause d'une erreur d'overlap. Il faut donc les séparer une deuxième fois, mais cette fois-ci tous les pavés, même les indéterminés, doivent être impactés par cet éclatement. Pour cela, rien de plus simple : on prend une origine (dans le sens où c'est un point suffisamment loin de tous les pavés et qui ne dépend pas de leur position à un instant t) et on calcule un vecteur de répulsion, obtenu en prenant une fraction du vecteur séparant le barycentre de chaque face de l'origine.

L'éclatement des pavés est enfin satisfaisant.

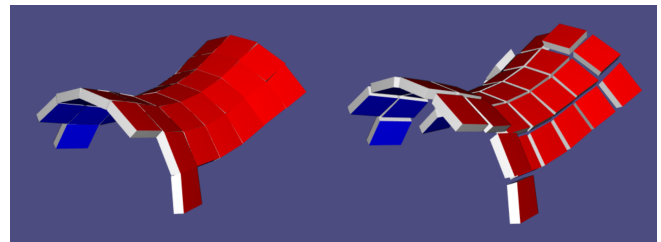


Fig. 8. À gauche, une coque hyperbolique. À droite, l'éclatement de ses pavés.

3.2 Regroupement des pavés

Dans un second temps, nous devons regrouper les pavés. Notre tâche est d'essayer de "les poser les uns sur les autres" le mieux possible.

Algorithme de rapprochement. L'algorithme est le suivant :

1. Initialiser un dictionnaire entre tags (c'est une sorte de "coloration des pavés" implémentée tout simplement par des entiers) et des ensembles de pavés (au départ, ces ensembles contiennent chacun un pavé différent).

2. Rapprocher les pavés les uns des autres.

3. Pour chacun des pavés en contact (test de collision réalisé avec la bibliothèque Bullet Physics), fusionner leurs ensembles et supprimer les tags devenus inutiles.

4. Retourner à l'étape 2 jusqu'à ce qu'il n'y ait plus qu'un seul tag (associé à un ensemble contenant tous les pavés de la coque).

Pour être précis, "rapprocher les pavés" consiste à les déplacer dans le sens inverse de l'éclatement convexe et à l'origine d'une distance ϵ . Plus ϵ est petit, mieux les contacts seront détectés (mais plus le nombre de rapprochements sera important). Dans ce projet, il est considéré que ϵ est suffisamment petit pour que les fonctionnalités de callback de Bullet Physics puisse permettre de revenir à la position des objets "juste avant la collision". En pratique, on prend $\epsilon = \frac{\text{dist}_{\text{eclatement}}}{\text{nb}_{\text{etapes}}}$, où $\text{dist}_{\text{eclatement}}$ correspond à la distance qui sépare la position de chaque pavé avant et après l'éclatement convexe et à l'origine et $\text{nb}_{\text{etapes}}$ correspond au nombre approximatif d'itérations de l'algorithme de rapprochement que l'on souhaite avant que tous les pavés soient en contact.

Optimisation des tests de collision. Un point de détail peut malgré cela causer un problème de complexité : afin vérifier les pavés qui sont en contact, une méthode naïve consiste à chaque étape à effectuer les tests de collision entre tous les pavés 2 à 2 (s'ils n'ont pas le même tag) (cf Figure 9.a.). Bullet Physics est assez subtil pour faire un test grossier afin de savoir si une collision peut exister (pour faire simple, il vérifie si les structures sont suffisamment proches pour s'intersecter) avant de vérifier la collision des structures qu'on lui donne (test coûtant beaucoup plus cher que le premier test grossier). Mais cette méthode naïve induit quand même un sur-coût linéaire en le nombre de pavés.

Une solution serait donc de considérer seulement les voisins du pavé examiné en se servant des itérateurs bien pratiques que peut fournir OpenMesh HalfEdge tels que FaceHalfEdgeIter qui itère

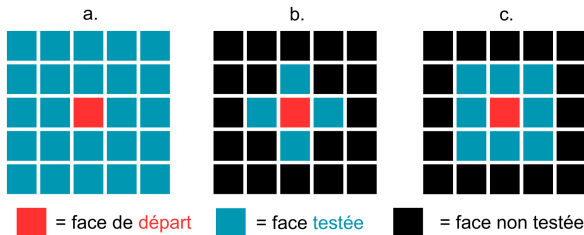


Fig. 9. a. méthode naïve, b. en se servant d'OpenMesh, c. en initialisant les voisins

sur les faces voisines de la face entrée en argument (cf Figure 9.b.). Cependant, cela soulève également un problème puisque cet itérateur accède seulement aux faces qui partagent une arête avec la face entrée en argument et non pas celles qui ne lui sont reliées que par un coin. On se retrouve donc avec des collisions potentielles non testées. Une dernière alternative consisterait à initialiser en début de programme pour chaque face la liste des "vrais voisins" (i.e. voisins reliés par au moins un coin) ce qui assurerait un sur-coût constant (cf Figure 9.c.). C'est cette dernière option qui a été adoptée dans l'application finale.

Le regroupement des pavés est enfin satisfaisant.

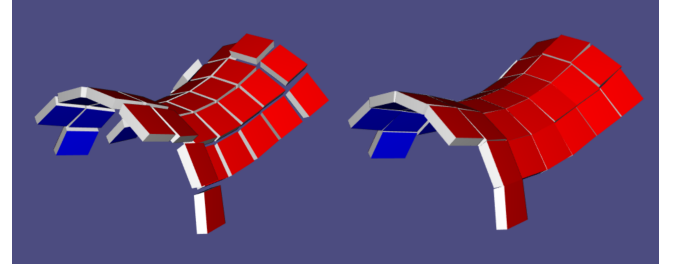


Fig. 10. À gauche, une coque hyperbolique éclatée. À droite, le regroupement de ses pavés.

3.3 Tests d'auto-soutenance

À présent que nous avons un maillage physiquement viable ainsi que la liste des points de collision qui relient chacun des pavés entre eux, il est maintenant possible de réaliser les tests de vérification d'auto-portance de notre coque.

Bullet Physics nous a permis à l'issue de la phase de regroupement des pavés de récupérer des données sur chacun des contacts entre voisins. En 3D, ces zones de contact sont des surfaces 2D. Cependant, pour simplifier la modélisation, le choix a été fait de considérer que les contacts s'effectuent selon des points ponctuels, qui correspondent aux barycentres des surfaces de contact.

Auto-soutenance 2D. En 2D, la situation est assez compréhensible. L'idée est d'équilibrer les forces appliquées en chacun des points de contact entre pavés voisins tout en conservant une égalité en norme des forces de réaction des pavés sur les pavés adjacents (voir la Figure 11), les forces de réaction devant s'effectuer vers l'extérieur des pavés (et non vers l'intérieur). À noter qu'il est toujours considéré que les conditions aux limites sont respectées (on peut imaginer un cadre comme dans la Figure 3 qui maintient les pavés sur les bords en place).

Exhiber une telle répartition des forces suffit pour affirmer que la structure s'auto-soutient.

Auto-soutenance 3D. Ici, l'enjeu réside dans les points de contact. Dans nos expérimentations, nous travaillons avec des pavés prenant la forme d'hexaèdres. Tandis qu'en 2D, il était facile d'avoir 2 points de contacts opposés pour chaque pavé (sauf ceux des bords) et donc des forces de réaction qui sont l'opposée l'une de l'autre, en 3D, on décompte dans le meilleur des cas quatre points de contacts par pavé (un par arête) et souvent moins. Les points de contacts sont la

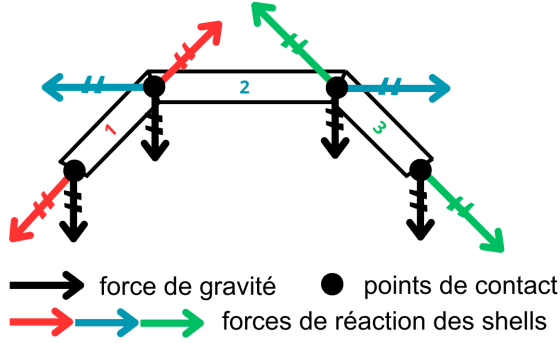


Fig. 11. Équilibre des forces sur une coque 2D. La somme des vecteurs des forces appliquées en chacun des points de contact doit être nulle et les forces opposées se doivent d'être de norme égale et de direction opposée pour que la structure s'auto-soutienne.

plupart du temps très exotiques (entre les coins de deux pavés en diagonale, plusieurs sur la même arête...).

Nous allons donc considérer que les forces de réaction peuvent s'effectuer entre chaque couple de points de collision voisins (c'est-à-dire qui partagent une face) selon la droite qui les séparent (voir Figure 12).

Formalisation mathématique. Considérons C l'ensemble des points de contact relevés à l'issue de la phase de regroupement. Supposons $x, y \in C$, soit $s_{x,y} (=s_{y,x})$ le pavé qu'ils partagent (s'il existe) et $v_x \subset C$ les points de contact voisins de x ($v_x = \{y \in C | s_{x,y} \text{ existe}\}$). Notons V l'ensemble des couples de points de contact voisins ($V = \{(x, y) \in C | y \in v_x \text{ ou } x \in v_y\}$) et $B \subset C$ les points de contact dont aucun des pavés n'est au bord du maillage (utile pour les conditions au limites). Enfin, notons $f_{x,y}$ la force de réaction exercée par le

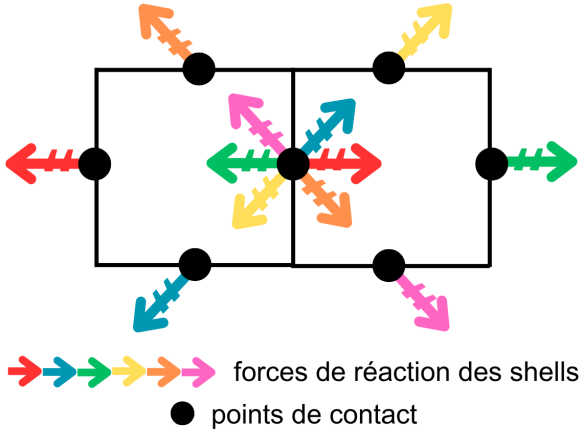


Fig. 12. Équilibre des forces sur une coque 3D. La somme des vecteurs des forces appliquées en chacun des points de contact doit être nulle et les forces voisines se doivent d'être de norme égale et de direction opposée pour que la structure s'auto-soutienne. Les forces de gravité (normales au dessin) n'ont pas été représentées sur le schéma.

pavé $s_{x,y}$ au point x et f_{g_x} la force de gravité imposée au point x (se référer à la Figure 13). Les conditions d'équilibre explicitées plus tôt reviennent ainsi à trouver une solution au système d'équations suivant (1):

$$\forall (x, y) \in V, f_{x,y} = -f_{y,x}$$

et

$$\forall x \in C \setminus B, \sum_{y \in v_x} f_{x,y} + f_{g_x} = 0$$

Pour donner plus de détail, $f_{g_x} = (0, 0, -m_x * g)$, sachant que $m_x = (m_{S_1} + m_{S_2})/2$ où S_1 et S_2 sont les pavés qui rentrent en collision en x et m_{S_1} et m_{S_2} leurs masses respectives. Les pavés sont considérés de masse volumique identique, seule leur volume varie.

Afin de faire en sorte que les forces de réaction soient dirigées vers l'extérieur des pavés et non pas vers l'intérieur, en notant $b_{x,y}$ le barycentre du pavé $s_{x,y}$, on peut rajouter les inéquations suivantes :

$$\forall (x, y) \in V$$

$$\langle f_{x,y}, x - b_{x,y} \rangle \geq 0$$

et

$$\langle f_{y,x}, y - b_{x,y} \rangle \geq 0$$

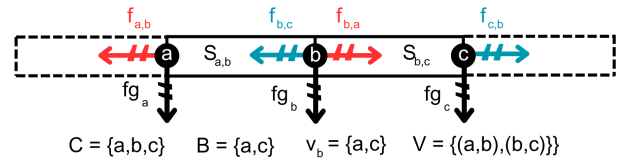


Fig. 13. Illustration des notations de la modélisation mathématique du problème d'auto-soutenance.

Sans rentrer dans les détails un peu trop techniques, cela revient à trouver une solution à un système matriciel $AX = B$. En notant $n = |V|$ et $m = |C \setminus B| = |C| - |B|$ et, on a $A \in (M)_{n,m}(\mathbb{R}^3)$, $X \in (M)_{m,1}(\mathbb{R}^3)$ et $B \in (M)_{n,1}(\mathbb{R}^3)$. D'un point de vue physique, le vecteur X correspond aux forces de réaction des pavés.

Pour prendre en compte les erreurs d'approximations (notamment lors des calculs de flottants par exemple) afin de déterminer s'il existe bel et bien une solution au système (1), on choisira de vérifier la validité du vecteur X (trouvé par une méthode LU par la bibliothèque Eigen, vraisemblablement approximé) via le test suivant :

$$|B - AX_{trouve}| < \epsilon$$

La dernière question en suspens est de choisir un ϵ pertinent.

4 CONCLUSION

4.1 Résultats

Avec la bibliothèque TileableShell (celle de l'algorithme de Chen et al.) étaient fournis un certain nombre de coques en maçonnerie sur laquelle les expérimentations précédemment explicitées ont pu être menées. On retrouve des erreurs d'approximation de l'ordre de 10^{-2} . Ces taux d'erreur semblent trop élevés pour n'être dus qu'à des arrondis de calculs de flottant. On pourrait donc conclure qu'il est possible que la structure n'est pas auto-soutenante car pour le graphe considéré, la structure n'est pas à l'équilibre. Néanmoins, en l'absence de coques dont on sait pertinemment qu'elles sont / ne sont pas auto-soutenantes (et auxquelles on aurait pu comparer les erreurs d'approximation), il est malheureusement difficile de se prononcer sur la signification de ces valeurs.

Coque testée	Fig 14.a.	Fig 14.b.	Fig 14.c.	Fig 14.d.
$ B - AX_{trouv} $	0.0140321	0.026569	0.0117408	0.0290251

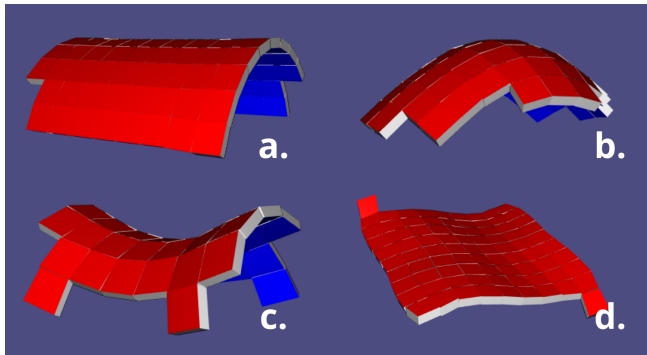


Fig. 14. Différentes coques en maçonnerie : a. Cylinder, b. Arch, c. Hyperbolic, d. MonkeySaddle

4.2 Défauts et pistes d'exploration

Tout d'abord, l'approche qui a été choisie ici pour repositionner les pavés des maillages en sortie de l'algorithme de Chen et al. est très empirique. Nous essayons des agencements mais rien ne suggère que ce sont les seuls possibles. En quelque sorte, nous ne pourrions pas sortir de "faux positifs" mais bien des "faux négatifs".

Les agencements sont d'autant plus compliqués à générer quand les points d'inflexion sont nombreux dans la surface 3D. On peut penser à la célèbre *Monkey Saddle* ou "Selle de singe" en anglais qui s'oppose diamétralement aux formes parfaitement convexes telles que la voûte simple (voir la Figure 14).

Ensuite, le développement du programme s'est centré autour de coques bien particulières, avec une certaine épaisseur et des pavages en forme d'hexaèdres. En ce sens, il ne peut pas être appliqué à tous les maillages possibles (avec des faces hexagonales, triangulaires ou autres...) et gagnerait à être généralisé, notamment au niveau de la visualisation (il faudrait pouvoir triangulariser tout type de Face en entrée) et des tests d'auto-soutenance.

Enfin, avec un peu plus de temps, il aurait été bienvenu de faire passer les coques en maçonnerie dont on connaît la stabilité (pourquoi pas les coques utilisées pour nos expérimentations avant qu'elle ne

soient passés par l'algorithme de Chen et al.) par notre algorithme d'auto-soutenance afin d'établir des valeurs de références d'erreurs d'approximation, dans le but de pouvoir exploiter nos résultats. On aurait aussi pu faire varier l'orientation des forces de gravité, pour tester la résistance aux orientations.

REFERENCES

- [1] M. Botsch, S. Steinberg, S. Bischoff, and L. Kobbelt. Openmesh: A generic and efficient polygon mesh data structure. <https://www.graphics.rwth-aachen.de/software/openmesh>, 2002.
- [2] Firstname Chen et al. Masonry shell structures with discrete equivalence classes. *ACM Transactions on Graphics*, 42(4), 2023.
- [3] Rulin Chen, Pengyun Qiu, Peng Song, Bailin Deng, Ziqi Wang, and Ying He. Masonry shell structures with discrete equivalence classes, 2023.
- [4] Erwin Coumans et al. Bullet physics library. <https://github.com/bulletphysics/bullet3>, 2003.
- [5] Fernando de Goes, Pierre Alliez, Houman Owjadi, and Mathieu Desbrun. On the equilibrium of simplicial masonry structures. *ACM Transactions on Graphics (SIGGRAPH)*, 32(4):93:1–93:10, 2013.
- [6] Chi-Wing Fu, Chi-Fu Lai, Ying He, and Daniel Cohen-Or. K-set tilable surfaces. *ACM Transactions on Graphics (SIGGRAPH)*, 29(4):44:1–44:6, 2010.
- [7] Gaël Guennebaud, Benoît Jacob, et al. Eigen v3. <http://eigen.tuxfamily.org>, 2010.
- [8] Mathieu Huard, Michael Eigensatz, and Philippe Bompas. Planar panelization with extreme repetition. In *Proc. Advances in Architectural Geometry 2014*, pages 259–279, 2015.
- [9] Alec Jacobson, Daniele Panozzo, et al. libigl: A simple C++ geometry processing library, 2018. <https://libigl.github.io/>.
- [10] Yang Liu, Hao Pan, John Snyder, Wenping Wang, and Baining Guo. Computing self-supporting surfaces by regular triangulation. *ACM Transactions on Graphics (SIGGRAPH)*, 32(4):92:1–92:10, 2013.
- [11] Masaaki Miki, Takeo Igarashi, and Philippe Block. Parametric self-supporting surfaces via direct computation of airy stress functions. *ACM Transactions on Graphics (SIGGRAPH)*, 34(4):89:1–89:12, 2015.
- [12] Daniele Panozzo, Philippe Block, and Olga Sorkine-Hornung. Designing unreinforced masonry models. *ACM Transactions on Graphics (SIGGRAPH)*, 32(4):91:1–91:11, 2013.
- [13] Adriaenssens Sigrid, Block Philippe, Veenendaal Diederik, and Williams Chris. *Shell Structures for Architecture: Form Finding and Optimization*. Routledge, 2014.
- [14] Mayank Singh and Scott Schaefer. Triangle surfaces with discrete equivalence classes, 2010.
- [15] Chengcheng Tang et al. Form-finding with polyhedral meshes made simple. *ACM Transactions on Graphics*, 33(4), 2014.
- [16] Firstname Wang et al. Design and structural optimization of topological interlocking assembling. *ACM Transactions on Graphics*, 38(6), 2019.
- [17] Emily Whiting et al. Structural optimization of 3d masonry buildings. *ACM Transactions on Graphics*, 31(6), 2012.
- [18] Emily Whiting, John Ochsendorf, and Frédo Durand. Procedural modeling of structurally-sound masonry buildings. *ACM Transactions on Graphics (SIGGRAPH Asia)*, 28(5):112:1–112:9, 2009.